

Documentation technique

Module `src/Analysis`

Application de statistiques sportives

Projet 1A — 2026

Ce document décrit l'ensemble des classes et fonctions du dossier `src/Analysis`, responsable du calcul des statistiques et de l'affichage des résultats pour chaque sport.

Table des matières

Architecture générale

Le dossier `src/Analysis` organise chaque sport dans un module dédié. Tous les modules suivent le même pattern structurel :

1. Des variables globales `_loader`, `_players`, `_matches` (etc.) initialisées à `None`.
2. Une fonction `_load()` qui charge les données **une seule fois** (cache dit *lazy loading*).
3. Des fonctions d'analyse qui appellent `_load()` en première instruction.
4. Une fonction `get_agenda_data()` qui formate les matchs dans un format standard pour l'agenda.

Le fichier `générique.py` est le **socle partagé** entre tous les modules : il fournit les fonctions d'affichage (bracket, classement), de formatage (roster, fiche joueur) et d'agrégation (agenda).

générique.py — Le moteur commun

Fichier : `src/Analysis/générique.py`

Ce fichier ne traite aucun sport en particulier. Il définit les briques réutilisables utilisées par l'ensemble des modules sportifs.

Classe `ResultatMatch`

Classe utilitaire qui encapsule le résultat d'un match à partir de deux scores.

Constructeur `__init__(equipe1, equipe2, score1, score2)`

Calcule automatiquement les attributs :

- `joue` (bool) — `True` si les deux scores sont disponibles et non `NaN`. La détection du `NaN` utilise la technique Python `score != score` (un float `NaN` n'est jamais égal à lui-même).
- `gagnant` — nom de l'équipe gagnante, ou `None` si le match n'est pas joué ou en cas d'égalité.
- `perdant` — nom de l'équipe perdante, ou `None`.

Méthode `label_equipe(equipe)`

Retourne la chaîne `"score1-score2"` si le match est joué, sinon `"???"`.

`formater_roster(df_joueurs, col_nom, ...)`

Formate le roster d'une équipe selon la spécification standard : `Rôle | Nom complet | Pseudo* | Nationalité | Date de naissance`.

- Les coaches apparaissent en tête de liste, puis les joueurs.
- La colonne `Pseudo` n'est incluse que si `est_esport=True`.
- Les colonnes absentes des données sont affichées comme `"N/A"`.
- Les dates sont formatées en `JJ/MM/AAAA`.

`_agreger_deux_manches(df, ...)`

Fonction privée. Pour les compétitions aller/retour (ex : Champions League), additionne les scores des deux matchs entre les mêmes équipes en un seul résultat. Utilise `tuple(sorted([eq1, eq2]))` pour identifier une paire indépendamment du sens domicile/extérieur.

`afficher_bracket(df, col_equipe1, col_equipe2, ...)`

Affiche un bracket d'élimination directe en **ASCII colorisé** dans le terminal. Construit une grille 2D de chaînes, puis des colonnes de connecteurs entre les rounds. Le chemin du gagnant est affiché en vert via les codes ANSI. Le paramètre `deux_manches` déclenche l'agrégation aller/retour avant affichage.

`_calculer_classement(df, ...)`

Fonction privée. Calcule un classement par points à partir d'un DataFrame de matchs.

- Règles : 3 pts victoire, 1 pt nul, 0 pt défaite.
- Tri : Points décroissants, puis différence de buts/scores décroissante.
- Rang : classement standard (ex-æquo = même rang, le suivant saute des places).

Retourne les colonnes Rang | Équipe | MJ | V | N | D | Pts | Diff | Winrate.

`_afficher_table_classement(df, top_qualifies, titre)`

Fonction privée. Affiche un tableau de classement avec une ligne séparatrice optionnelle après les `top_qualifies` premières équipes (ex : TOP 2).

`afficher_classement(df, col_equipe1, col_equipe2, ...)`

Orchestre `_calculer_classement` et `_afficher_table_classement`. Si `col_groupe` est fourni, itère sur chaque groupe et affiche un tableau de classement par groupe.

`fiche_joueur(df_joueurs, col_nom, nom_joueur, ...)`

Retourne la fiche complète d'un joueur sous forme Statistique | Valeur.

- Recherche insensible à la casse, accepte les noms partiels.
- Lève `ValueError` si aucun résultat.
- Lève `ValueError` si plusieurs joueurs correspondent (liste les correspondances pour affiner la recherche).
- Les colonnes sont renommées selon le dictionnaire `col_labels`.

`lister_joueurs(df_joueurs, col_nom, ...)`

Retourne la liste simple de tous les joueurs d'un sport, triée par nom. Colonnes : Nom (et Équipe si la colonne est fournie).

`agenda_recents(sources, n=10)`

Agrège les DataFrames de plusieurs sports au format standard Sport | Date | Équipe 1 | Équipe 2 | Score 1 | Score 2, trie par date décroissante, et retourne les `n` matchs les plus récents.

`badminton.py`

Fichier : `src/Analysis/badminton.py`

`_load()`

Cache lazy : instancie `BadmintonLoader` et charge joueurs + matchs une seule fois dans les variables globales du module.

`_compter_jeux(row)`

Fonction privée. Parse les colonnes `game_1_score`, `game_2_score`, `game_3_score` au format "21-15" et retourne un tuple (`wins_joueur1`, `wins_joueur2`). Gère les sets non joués (NaN ou chaîne vide).

`fiche_joueur_badminton(nom)`

Délègue à `générique.fiche_joueur`. Affiche nom, pays, continent.

`bilan_joueur_badminton(nom)`

Calcule victoires et défaites en cherchant le joueur comme `player_1` ET comme `player_2` dans les matchs. Utilise `_compter_jeux` pour déterminer le gagnant. Retourne un `DataFrame Statistique | Nom du joueur`.

`bracket(tournament_name)`

Filtre les matchs d'un tournoi sur les rounds bracket (Round of 32 jusqu'à Final), calcule les scores avec `_compter_jeux`, puis délègue à `générique.afficher_bracket`.

`get_agenda_data()`

Retourne les matchs au format standard agenda, avec le nombre de jeux gagnés comme score.

`basketball.py`

Fichier : `src/Analysis/basketball.py`

`_load()`

Cache lazy : charge joueurs, équipes et matchs NBA.

`top_equipes_offensives(n=10)`

Fusionne les colonnes `pts_home` et `pts_away` en un format neutre `team_id / pts`, calcule la moyenne de points par match par équipe, trie par ordre décroissant.

`stats_equipe(team_name)`

Recherche l'équipe par nom complet, abréviation ou surnom. Fusionne les matchs à domicile et à l'extérieur (en supprimant les suffixes `_home` et `_away`). Calcule la moyenne de 12 statistiques (points, rebonds, passes, tirs, lancers francs, interceptions, contres, pertes de balle).

`roster_equipe(team_name)`

Appelle `BasketballLoader.get_team_roster()` puis délègue à `générique.formater_roster`.

`classement_defensif(n=10)`

Calcule les points encaissés, blocks et interceptions moyens par équipe. Trie par points encaissés croissants (moins on en prend, meilleure est la défense).

`fiche_joueur_basketball(nom)`

Délègue à `générique.fiche_joueur` avec les labels en français.

`liste_joueurs(equipe=None)`

Liste tous les joueurs NBA, ou uniquement ceux d'une équipe si un nom est précisé.

`get_agenda_data()`

Mappe les identifiants d'équipes (`team_id`) vers les noms complets, retourne le format standard agenda.

`classement_points(saison=None, top_qualifies=None)`

Filtre sur Regular Season, mappe les identifiants d'équipes vers les noms, délègue à `générique.afficher_classement`.

`chess.py`

Fichier : `src/Analysis/chess.py`

`_load()`

Cache lazy : charge joueurs et matchs d'échecs.

`classement_elo(mode="standard", n=20)`

Accepte les modes `"standard"`, `"rapid"` ou `"blitz"`. Mappe vers la colonne Elo correspondante, supprime les NaN, trie et retourne les N premiers joueurs avec leur fédération et titre FIDE.

`fiche_joueur_chess(nom)`

Délègue à `générique.fiche_joueur` avec labels : Nom, Fédération, Titre FIDE, Elos standard/rapid/blitz.

bilan_joueur(player_name)

Cherche le joueur comme `player_1` et comme `player_2`. Utilise `pd.to_numeric(..., errors="coerce")` pour convertir les scores (pouvant contenir "Bye"). Un score de 1 = victoire, 0.5 = nul, 0 = défaite. Retourne aussi le score total (victoires + 0,5 × nuls), fidèle aux règles des échecs.

stats_par_titre()

Groupe les joueurs par titre FIDE (GM, IM, etc.) et calcule le nombre de joueurs ainsi que l'Elo moyen dans chaque catégorie (standard, rapid, blitz).

cs2.py

Fichier : `src/Analysis/cs2.py`

_load()

Cache lazy : charge joueurs, coaches, équipes et matchs CS2.

classement()

Calcule victoires et matchs joués par équipe sur l'ensemble de la compétition. Détermine le gagnant de chaque match par comparaison directe des scores.

stats_equipe(team_name)

Calcule matchs joués, victoires, défaites, et **maps gagnées et perdues** (cumul des scores). La distinction matchs/maps est propre au CS2 : un score de 2-1 signifie 2 maps à 1.

roster_equipe(team_name)

Combine joueurs et coaches filtrés par équipe, délègue à `générique.formater_roster` avec `est_esport=True` (inclut la colonne Pseudo).

fiche_joueur_cs2(nom)

Délègue à `générique.fiche_joueur`.

liste_joueurs(equipe=None)

Liste tous les joueurs CS2, filtrables par équipe.

get_agenda_data()

Retourne le format standard agenda avec les scores en nombre de maps.

bracket()

Filtre les matchs de stage "PlayOffs" et délègue à `générique.afficher_bracket` avec `deux_manches=False` (pas d'aller/retour en CS2).

`classement_stages(top_qualifies=None)`

Affiche un classement par phase de groupes (stages 1, 2, 3). Crée une colonne `stage_label` et exclut les PlayOffs avant de déléguer à `générique.afficher_classement`.

[`football\_cl.py`](#)

Fichier : `src/Analysis/football_cl.py`

`_load()`

Cache lazy : charge joueurs, équipes et matchs Champions League.

`meilleurs_buteurs(n=10)`

Trie les joueurs par buts et calcule le pourcentage de tirs cadrés (`on_target` / `total_attempts`).

`meilleurs_passeurs(n=10)`

Trie les joueurs par nombre de passes décisives.

`stats_equipe(team_name)`

Agrège les statistiques de tous les joueurs d'un club (somme totale) en buts, passes décisives, minutes jouées, cartons jaunes et cartons rouges.

`stats_joueur(player_name)`

Retourne toutes les statistiques individuelles d'un joueur de la Champions League (buts, passes, dribbles, tacles, cartons, passes tentées/réussies, minutes).

`stats_gardiens(n=10)`

Filtre les joueurs dont la position contient `Goalkeeper`, `GK` ou `Gardien`, trie par nombre d'arrêts. Retourne arrêts, buts encaissés, clean sheets, pénaltys arrêtés.

`fiche_joueur_fcl(nom)`

Délègue à `générique.fiche_joueur` avec un dictionnaire de labels très détaillé (20 colonnes).

`liste_joueurs(club=None)`

Liste tous les joueurs de la CL, filtrables par club.

`classement_groupes(top_qualifies=2)`

Filtre les matchs de phase "group" et délègue à `générique.afficher_classement` avec `col_groupe="group"`.

`get_agenda_data()`

Retourne le format standard agenda.

`bracket()`

Filtre les matchs de phase "knockout" et délègue à `générique.afficher_bracket` avec `deux_manches=True` (aller/retour en Champions League).

[lol.py](#)

Fichier : `src/Analysis/lol.py`

`_load()`

Cache lazy : charge joueurs, coaches, équipes et matchs LoL.

`stats_equipe(team_name)`

Cherche l'équipe dans les colonnes `team_blue` et `team_red`, fusionne les matchs en renommant les suffixes `_team_blue` et `_team_red`. Calcule la moyenne de kills, assists, deaths, gold, turrets, dragons, barons, et la **durée moyenne en minutes**.

`champions_picks_bans(n=10)`

Identifie toutes les colonnes commençant par `pick_` et `ban_`, aplatit leurs valeurs, compte les occurrences de chaque champion. Retourne un top N par picks et bans.

`roster_equipe(team_name)`

Combine joueurs et coaches, délègue à `générique.formater_roster` avec `est_esport=True`.

`fiche_joueur_lol(nom)`

Délègue à `générique.fiche_joueur`.

`liste_joueurs(equipe=None)`

Liste les joueurs LoL, filtrables par équipe.

`get_agenda_data()`

Le score est 1 pour le gagnant, 0 pour le perdant.

`classement_points(top_qualifies=None)`

Crée des colonnes `score_blue` et `score_red` à partir de la colonne `winner`, puis délègue à `générique.afficher_classement`.

`starcraft2.py`

Fichier : `src/Analysis/starcraft2.py`

`_load()`

Cache lazy : charge joueurs et matchs SC2.

`fiche_joueur_sc2(nom)`

Délègue à `générique.fiche_joueur`. Inclut la colonne `race` (Terran / Zerg / Protoss), spécifique à StarCraft II.

`bilan_joueur_sc2(nom)`

Cherche le joueur comme `player_1` et comme `player_2`. Compare directement `score_player_1` > `score_player_2` pour déterminer le gagnant.

`get_agenda_data()`

Retourne le format standard agenda avec les scores en nombre de maps.

`tennis.py`

Fichier : `src/Analysis/tennis.py`

`_load()`

Cache lazy : charge les quatre tables ATP et WTA.

`_matches(circuit) / _players(circuit)`

Fonctions helpers privées qui retournent la bonne table selon "ATP" ou "WTA".

`classement_victoires(circuit="ATP", n=20)`

Compte victoires (`winner_id`) et défaites (`loser_id`) par identifiant joueur, joint les noms complets, trie et retourne les N meilleurs.

`stats_joueur(player_name, circuit="ATP")`

Retourne victoires, défaites, pourcentage de victoires, nombre de tournois, et la durée moyenne d'un match si la colonne `minutes` est disponible.

`fiche_joueur_tennis(nom, circuit="ATP")`

Délègue à `générique.fiche_joueur`.

`liste_joueurs(circuit="ATP")`

Liste tous les joueurs du circuit sélectionné.

`_compter_sets(score)`

Fonction privée. Parse une chaîne de score tennis comme "7-6(5) 6-4". Utilise `re.sub(r'\(\\d+\\)', '', ...)` pour supprimer les scores de tie-break avant de compter les sets.

`_standardiser_tennis(matches, players, label)`

Fonction privée. Mappe les `winner_id` et `loser_id` vers les noms complets, calcule les sets via `_compter_sets`, retourne le format agenda standard.

`get_agenda_data()`

Concatène les matchs ATP et WTA standardisés. Le score correspond aux sets gagnés (gagnant / perdant).

`bracket(tourney_name, circuit="ATP")`

Filtre les matchs d'un tournoi, mappe les identifiants en noms, calcule les sets, filtre sur les rounds bracket (R128 jusqu'à F), délègue à `générique.afficher_bracket`.

[`volleyball.py`](#)

Fichier : `src/Analysis/volleyball.py`

`_load()`

Cache lazy : charge les sept tables de volleyball.

`_matches(genre) / _players(genre) / _country_name(code)`

Helpers privés pour sélectionner la bonne table par genre ("hommes" ou "femmes"), et pour résoudre un code IOC en nom de pays complet.

`classement(genre="hommes")`

Calcule victoires, défaites et **sets gagnés** (critère de départage en volleyball). Enrichit le résultat avec les noms complets de pays via la table `_countries`.

`bilan_equipe(team_code, genre="hommes")`

Calcule le bilan d'une équipe identifiée par un code IOC (ex : "FRA"), avec sets gagnés et sets perdus.

`roster_equipe(team_code, genre="hommes")`

Filtre par code IOC exact (pas de recherche partielle, les codes sont standardisés), combine joueurs et coachs, délègue à `générique.formater_roster`.

`fiche_joueur_volleyball(nom, genre="hommes")`

Délègue à `générique.fiche_joueur`. Inclut `height`, `birth_place` et `nickname`.

```
liste_joueurs(genre="hommes", equipe=None)
```

Liste les joueurs d'un genre, filtrables par code pays.

```
get_agenda_data()
```

Mappe les codes IOC vers les noms complets de pays. Concatène les matchs hommes et femmes.

```
classement_groupes(genre="hommes", top_qualifies=2)
```

Extrait la lettre de poule depuis la colonne `stage` via le regex `Pool ([A-Z])`, puis délègue à `générique.afficher_classement`.

Synthèse des patterns communs

Pattern	Description
Lazy loading	Les données ne sont chargées qu'au premier appel de <code>_load()</code> , puis mises en cache dans les variables globales du module
<code>get_agenda_data()</code>	Chaque module expose cette fonction au format standard, permettant l'agrégation multi-sport via <code>générique.agenda_recents()</code>
Délégation à générique	Les fonctions d'affichage (<code>bracket</code> , <code>classement</code> , <code>fiche</code> , <code>roster</code>) sont centralisées dans <code>générique.py</code> et appelées par tous les modules
Recherche partielle	Toutes les recherches par nom acceptent des noms partiels et sont insensibles à la casse
<code>ValueError</code> explicite	Si aucun résultat ou si la recherche est ambiguë, une <code>ValueError</code> est levée avec un message lisible